

**BACHELOR PROJECT — DFKI**

# Minimal Dependency Dataset Library for PyTorch

**Pascal C. Nague** · RPTU Kaiserslautern

Supervisor · **Dr. Krzysztof Adamkiewicz**

*By Nague*

# Motivation

---

## Problem

Deep learning performance is not only limited by the GPU.

Existing solutions require heavy dependencies, rigid formats, or invasive code changes.

---

## Consequence

Slow data loading leaves expensive hardware idle.

---

## Goal

Explore a lightweight alternative with minimal integration friction.

# Project Goal & Research Questions

## Project Goal

- Simple integration into PyTorch training loops
- Parquet-backed dataset storage
- Concurrent data loading
- Performance instrumentation
- Benchmark comparison with PyTorch DataLoader

## Research Questions

1. Can throughput stay competitive?
2. How does it scale with more workers?
3. Where are the bottlenecks?
4. Which optimizations help most?
5. Does it generalize to other datasets?

📄 The prototype focuses mainly on image datasets, while the DataLoader architecture itself is designed to work with arbitrary dataset samples.

# Project Phases

## 1 Phase 1

GPU benchmarking to understand the training environment.

## 2 Phase 2

Storage format evaluation across file types.

## 3 Phase 3

Custom DataLoader prototype with concurrent workers.

## 4 Phase 4

Instrumentation and benchmarking of pipeline behavior.

## 5 Phase 5

Optimization of decoding, staging, and queues.

## 6 Phase 6

External validation on Food-101.

# Architecture & ParquetDataset

Parquet File

ParquetDataset

Worker Threads

Chunked Staging Queue

- LockFreeSampler distributes samples
- JPEG decode with torchvision
- Resize while still uint8
- Float conversion & normalization delayed to batch collation

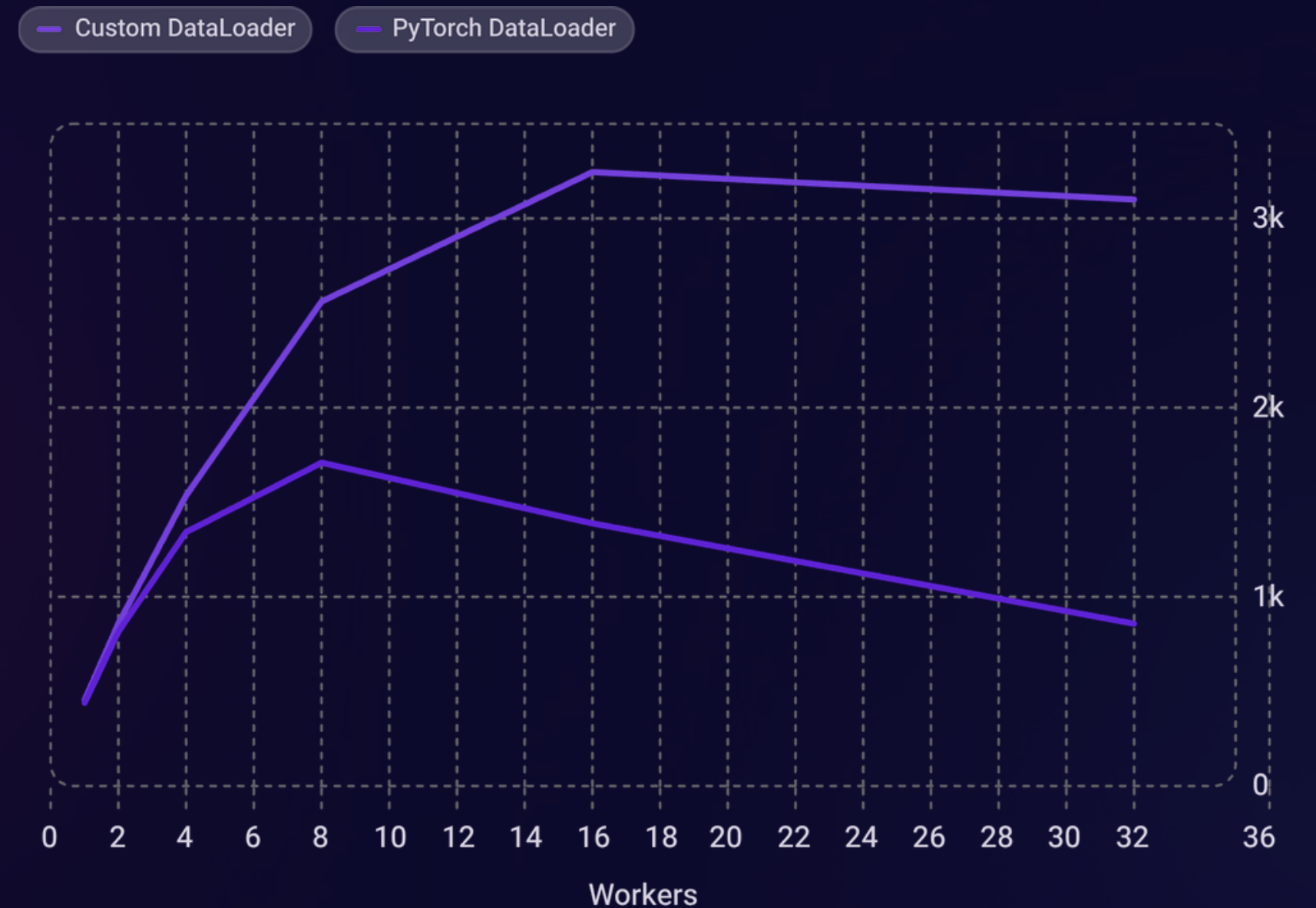
❏ Current Parquet loading is eager, not lazy.

# Benchmark Setup & Main Results

## Benchmark Configuration

- **Dataset:** Image dataset stored in Parquet
- **Samples:** 9,984
- **Batch size:** 256
- **Workers:** 1, 2, 4, 8, 16, 32
- **Repetitions:** 3
- **Metric:** Samples/second

Compared `torch.utils.data.DataLoader` vs. `minimal_dataset.DataLoader` using the same dataset and preprocessing pipeline.



✅ **Peak result:** The custom DataLoader reached **3,245 samples/s at 16 workers** — outperforming PyTorch DataLoader under this benchmark configuration.

# Optimization Impact

1,517

Before  
samples/s

3,245

After  
samples/s

2.1x

Improvement  
Throughput

□ The largest contribution was not one trick, but **systematic measurement and iterative optimization.**

## Key Optimizations

→ **Chunked staging** — reduced queue contention

→ **Batch-level float conversion** — delayed normalization

→ **Optimized JPEG decode + resize path**

→ **Better instrumentation** — precise bottleneck identification

# External Validation: Food-101

Tested the library on Food-101 to check whether benchmark results generalize beyond the initial dataset.

## Setup

- 10,000 Food-101 training images
- Parquet storage
- Image sizes: 224×224 and 64×64
- Batch sizes: 64 and 256
- Same comparison against PyTorch DataLoader

## Results

### 224×224

Custom DataLoader was faster than PyTorch from 1 to 4 workers. PyTorch scaled better at higher worker counts.

### 64×64

Both implementations became faster. Custom DataLoader scaled farther than at 224×224, but PyTorch still achieved higher peak throughput.



**Performance advantage is workload-dependent.**

# Limitations, Future Work & Contributions

## CURRENT LIMITATIONS

- Built-in datasets mainly image-focused
- Eager Parquet loading
- Single-orchestrator design may contribute to scaling limitations and requires further profiling
- Benchmark measures data-loading throughput not full training throughput
- Exact Food-101 bottleneck not fully isolated

## FUTURE WORK

- Generic decoder abstraction
- Text and video support
- Lazy / row-group Parquet loading
- Better distributed training support
- Further profiling of Food-101 scaling
- Full training throughput and GPU utilization

## MAIN CONTRIBUTIONS

- Lightweight PyTorch-compatible API
- Parquet-backed image dataset
- Custom concurrent DataLoader
- Instrumentation and benchmarking
- PyPI package with external validation

# Conclusion

1. The custom DataLoader achieved strong performance under the original benchmark configuration.
2. Food-101 showed that the result is workload-dependent.
3. The project provides both a working library and an experimental basis for further data-loading research.

The most important lesson is that performance engineering requires not only optimization, but also honest measurement and careful interpretation.

# Questions?

GitHub

**minimal-dataset-pytorch**

PyPI

**minimal-dataset-pytorch**



 [www.linkedin.com](https://www.linkedin.com)



**LinkedIn for Feedback**



*By Nagne*